

instances. We do not have to manually remember the previous count. We have provided the end state and everything else will be handled by Terraform itself. This demonstrates all possible scenarios when manual interventions occur, such as slow speed, prone to human error, etc.

Advantages of incorporating IaC using Terraform

1. **Prevents configuration drift:** Terraform, being provisioning software, binds you to make changes in your container and only then deploy the new ones across every server. This separates server configuration from any dependency, resulting in identical instances across our infrastructures.
2. **Easy collaboration:** The terraform registry (Terraform's central registry version control) enables teams to collaborate on infrastructure.
3. **No separate documentation needed:** The code written for infrastructure will become your documentation. By looking at the script, thanks to its procedural nature, we can figure out what's currently deployed and its configuration.
4. **Flexibility:** Terraform not only handles IaaS (AWS, Azure, etc) but also PaaS (SQL, NodeJS). It can also

```
1 ## Procedural
2 ec2:
3   count: 5
4   image: ami-v1
5   instance_type: t2.micro
```

```
1 ## Declarative
2 resource "aws_instance" "example" {
3   count = 5
4   ami = "ami-v1"
5   instance_type = "t2.micro"
```

Figure 8a and 8b: Sample code for instantiating five instances

```
1 ## Procedural
2 ec2:
3   count: 10
4   image: ami-v1
5   instance_type: t2.micro
```

```
1 ## Declarative
2 resource "aws_instance" "example" {
3   count = 10
4   ami = "ami-v1"
5   instance_type = "t2.micro"
```

Figure 9a and 9b: Sample code for instantiating ten instances

store local variables such as cloud tokens and passwords in encrypted form on the terraform registry.

5. **Masterless:** Terraform is masterless, by default, i.e., it does not need a master node to keep track of all configuration and distributing updates. This saves the extra infrastructure and maintenance costs we'd have to incur in maintaining an extra master node. Terraform directly uses the cloud providers' API, thus

saving extra infrastructure costs and other overheads.

Terraform is an open source tool that helps teams manage infrastructure in an efficient, automated and reusable manner. It has a simple modular syntax and supports multi-cloud infrastructure configuration. Enterprises can use Terraform in their DevOps methodology to construct, modify, manage and deliver infrastructure at a faster pace with less manual intervention. **END** 🐧

By: Vaibhav Aggarwal and Prof. B. Thangaraju

The authors are associated with the open source technology lab in the International Institute of Information Technology, Bengaluru.

OSFY Magazine Attractions During 2020-21

MONTH	THEME
March 2020	Mobile/Web App Development and Optimisation
April 2020	Cybersecurity, Open Source Firewall, Network Security and Monitoring
May 2020	AI, Deep Learning and Machine Learning
June 2020	DevOps Special
July 2020	Blockchain and Open Source
August 2020	Database Management and Optimisation
September 2020	Open Source and IoT
October 2020	Cloud Special: BigData, Hadoop, PaaS, SaaS, IaaS and Cloud
November 2020	Open Source on Windows
December 2020	Open Source Programming (Languages and Tools)
January 2021	Data Security, Storage and Backup
February 2021	Best in the World of Open Source (Tools and Services)